7.4 Standard methods of solution

The ability to repeat existing methods is very important in the design of algorithms; when an algorithm is turned into a program the same methods may be repeated many thousands of times.

You need to be able to use and understand these standard methods used in algorithms:

- Totalling
- Counting
- Finding maximum, minimum, and average (mean) values
- Searching using a linear search
- Sorting using a bubble sort.

All the standard methods of solution are shown as pseudocode algorithms and will be used to practise program writing in the next chapter.

7.4.1 Totalling

Totalling means keeping a total that values are added to. For example, keeping a running total of the marks awarded to each student in a class.

```
Total ← 0
FOR Counter ← 1 TO ClassSize
    Total ← Total + StudentMark[Counter]
NEXT Counter
```

Initialising `Total` to zero

Totalling the marks in an array called `StudentMark`

7.4.2 Counting

Keeping a count of the number of times an action is performed is another standard method. For example, counting the number of students that were awarded a pass mark:

```
PassCount ← 0
FOR Counter ← 1 TO ClassSize
    INPUT StudentMark
    IF StudentMark > 50
        THEN
            PassCount ← PassCount + 1
NEXT Counter
Count ← Count + 1
```

Initialising `PassCount` to zero

Counting the number of passes

Counting is also used to count down until a certain value is reached, for example, checking the number of items in stock in a supermarket:

```
:
NumberInStock ← NumberInStock - 1
IF NumberInStock < 20
    THEN
        CALL Reorder()
:
```

Counting down items in stock

7.4.3 Maximum, minimum and average

Finding the largest and smallest values in a list are two standard methods that are frequently found in algorithms, for example, finding the highest and lowest mark awarded to a class of students.

```
MaximumMark ← 0
MinimumMark ← 100
FOR Counter ← 1 TO ClassSize
  IF StudentMark[Counter] > MaximumMark
    THEN
      MaximumMark ← StudentMark[Counter]
  ENDIF
  IF StudentMark[Counter] < MinimumMark
    THEN
      MinimumMark ← StudentMark[Counter]
  ENDIF
NEXT Counter
```

Initialising maximum to the lowest mark possible

Initialising minimum to the highest possible

Calls data from an array (see Chapter 8) called `StudentMark`)

Replacing the maximum mark with a higher mark

Replacing the minimum mark with a lower mark

If the largest and smallest values are not known, an alternative method is to set the maximum and minimum values to the first item in the list.

For example, using this method to find the highest and lowest mark awarded to a class of students.

```
MaximumMark ← StudentMark[1]
MinimumMark ← StudentMark[1]
FOR Counter ← 2 TO ClassSize
  IF StudentMark[Counter] > MaximumMark
    THEN
      MaximumMark ← StudentMark[Counter]
  ENDIF
  IF StudentMark[Counter] < MinimumMark
    THEN
      MinimumMark ← StudentMark[Counter]
  ENDIF
NEXT Counter
```

Initialising minimum and maximum to the first mark

Starting the loop at the second position in the list.

Calculating the average (mean) of all the values in a list is an extension of the totalling method, for example, calculating the average mark for a class of students.

```
Total ← 0
FOR Counter ← 1 TO ClassSize
  Total ← Total + StudentMark[Counter]
NEXT Counter
Average ← Total / ClassSize
```

Calculating the average from the total after the loop has been completed

7.4.4 Linear search

A search is used to check if a value is stored in a list, performed by systematically working through the items in the list. There are several standard search methods, but you only need to understand one method for IGCSE Computer Science. This is called a **linear search**, which inspects each item in a list in turn to see if the item matches the value searched for.

For example, searching for a name in a class list of student names, where all the names stored are different:

```
OUTPUT "Please enter name to find "
INPUT Name
Found ← FALSE
Counter ← 1
REPEAT
  IF Name = StudentName[Counter]
```

Setting a variable, `Found`, as a flag, using `TRUE` and `FALSE` to indicate if the name has been found or not

Checking if the name input matches a name in the list

```
  THEN
     Found ← TRUE
    ELSE
      Counter ← Counter + 1
  ENDIF
UNTIL Found OR Counter > ClassSize
IF Found
  THEN
    OUTPUT Name, " found at position ",
Counter, " in the list."
  ELSE
    OUTPUT Name, " not found."
ENDIF
```

Setting the flag to TRUE when a match is found

Stopping the search when a match is found or the whole list has been searched

Outputting the position in the list when a match is found

In this example, the search checks how many people chose ice cream as their favourite dessert, where several values in the list can be the same.

```
ChoiceCount ← 0
FOR Counter ← 1 TO Length
IF "ice cream" = Dessert[Counter]
  THEN
   ChoiceCount ← ChoiceCount + 1
NEXT Counter
OUTPUT ChoiceCount, " chose ice cream
as their favourite dessert."
```

Checking ice cream has been chosen

Checking every item in the list

7.4.5 Bubble sort

Lists can be more useful if the items are sorted in a meaningful order. For example, names could be sorted in alphabetical order, or temperatures could be sorted in ascending or descending order. There are several standard sorting methods available, but you only need to understand one method for IGCSE Computer Science.

This method of sorting is called a **bubble sort**. Each element is compared with the next element and swapped if the elements are in the wrong order, starting from the first element and finishing with next-to-last element. Once it reaches the end of the list, we can be sure that the last element is now in the correct place. However, other items in the list may still be out of order. Each element in the list is compared again apart from the last one because we know the final element is in the correct place. This continues to repeat until there is only one element left to check or no swaps are made.

For example, the bubble sort algorithm can be used to sort a list of ten temperatures stored in the array, **`Temperature[]`**, into ascending order. It could be written in pseudocode as:

```
First ← 1
Last ← 10
REPEAT
  Swap ← FALSE
  FOR Index ← First TO Last - 1
    IF Temperature[Index] > temperature[Index + 1]
      THEN
        Temp ← Temperature[Index]
        Temperature[Index] ← Temperature[Index + 1]
        Temperature[Index + 1] ← Temp
        Swap ← TRUE
    ENDIF
  NEXT Index
  Last ← Last - 1
UNTIL (NOT Swap) OR Last = 1
```

The `IF..THEN` condition checks if temperatures are in ascending order and swaps them if they are not, using the Temp variable (short for temporary)

7.5 Validation and verification

In order for computer systems to only accept data inputs that are reasonable and accurate, every item of data needs to be examined before it is accepted by the system.

Two different methods, with very similar sounding names, are used. For data entry, validation ensures that only data that is reasonable is accepted. Verification is used to check that the data does not change as it is being entered.

7.5.1 Validation

Validation is the automated checking by a program that data is reasonable before it is accepted into a computer system. When data is validated by a computer system, if the data is rejected a message should be output explaining why the data was rejected and another opportunity given to enter the data.

There are many different types of validation checks including:

- **range checks**
- **length checks**
- **type checks**
- **presence checks**
- **format checks**
- **check digits.**